

## Comprehensive Project Summary (Technical Details Included)

For a long time, the Aerodynamics Department at FSUK relied on manual sketching in SolidWorks for airfoil design. This empirical process lacked standardization and reproducibility, creating a vicious cycle of manual trial-and-error and untrustworthy data.

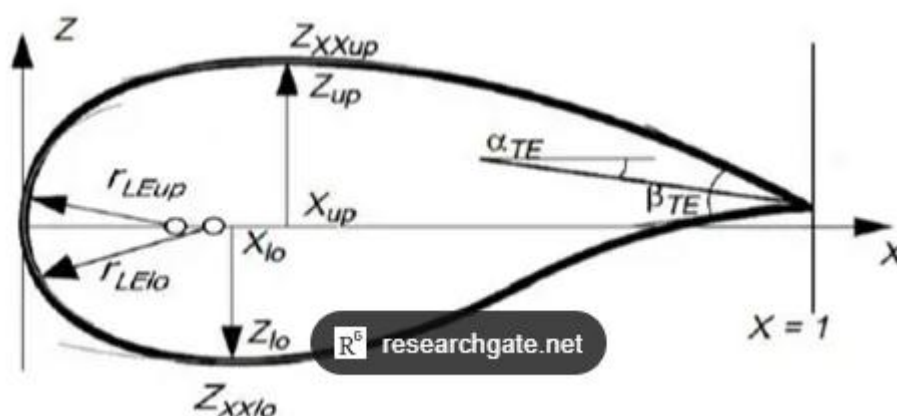
To fundamentally resolve these pain points and upgrade the design workflow toward a data-driven, automated, and parametric framework, this project systematically developed a high-quality airfoil database, automated simulation pipelines, parametric tools, and a visualization platform. The project covers the entire chain: data acquisition, quality control (QC), geometric generation, simulation feedback, and engineering export.

---

### Phase I: NACA Aerodynamic Database (From 0 to 1)

- **V1.0: PARSEC Parametric Experimentation (Airfoil database 1.0, 1.1)**

Initial attempts utilized the PARSEC parameterization method, using AI to generate parameters combined with Python scripts to control XFOIL iterations (xfoil\_gai.py). PARSEC describes airfoils via 11 geometric parameters (e.g., leading-edge radius, crest location). However, AI-generated parameters were often unstable, resulting in deformed geometries (sunken leading edges or cracked trailing edges) that caused XFOIL to hang or diverge. The PARSEC approach was subsequently abandoned in favor of the more stable NACA system.



- **V2.0: Automated Batch Simulation (Airfoil database 1.1, 1.2)**

The workflow shifted to the XFOIL internal NACA generator, which ensures smooth, standardized geometry and natural convergence. Technically, I implemented a Python script utilizing a configuration-file logic (similar to OpenFOAM) to decouple

code from data (camber, thickness, Reynolds number, AoA, etc.).

```
}
naca_settings = {
    "max_camber": [True, 4, 6, 1], #[Iterate?True/False, max_val, min_val, step]
    "max_camber_position": [True, 40, 70, 10],
    "thickness": [True, 8, 13, 1.0],
    # --- 计算设置 -----
    "reynold": [True, 5e5, 1e6, 2.5e5],
    "alpha": [True, 0, 10, 0.1],
    "panels": [False, 300, 400, 50],
}

def gen_cases(settings):
    cases = {}
    keys = list(settings.keys())
    count = 1

    for i in range(len(keys)):
        content = settings[keys[i]]
        if content[0]:
            cases[keys[i]] = np.arange(content[1], content[2] + content[3]/10, content[3]).tolist()
        else:
            cases[keys[i]] = [content[1]]
        count = count * len(cases[keys[i]])

    print(cases)
    return cases, count
```

By traversing all parameter combinations via Cartesian product, the script automatically generated massive test cases and parsed output data (C\_L, C\_D, L/D, and separation points). However, this version lacked error handling, and the raw dataset contained high levels of non-physical "garbage" data.

---

## Phase II: Data Cleaning and Quality Control

- **V1.0: Stability Optimization**

To address XFOIL hangs and script crashes, I introduced a timeout termination mechanism using subprocess (10s limit) to prevent process deadlocks.

```
try:
    # 3. 执行
    proc = subprocess.run(
        f"{xfoil_path} < {input_file}",
        #shell=True,
        input=input_str,
        capture_output=True, text=True, encoding='utf-8', errors='ignore', timeout=10
    )

    stdout = proc.stdout
except subprocess.TimeoutExpired:
    # 4. 如果超时，打印警告并记录，准备进入下一次循环
    print(f"⚠ Subprocess timed out\n")
    status = False
    return None, None, status
# 注意: subprocess.run 在超时时会自动 kill 掉子进程
```

I also added convergence checks to skip non-converged cases and implemented

memory caching with batch-writing to disk to reduce I/O overhead.

```
# 4. 定期保存到 CSV
if (i + 1) == count:
    save_to_csv(results_buffer, csv_file)
elif len(results_buffer) >= 20:
    save_to_csv(results_buffer[:-1], csv_file)
    results_buffer = [results_buffer[-1]] # 清空缓存
    print("\n -----AUTO SAVED-----\n")
```

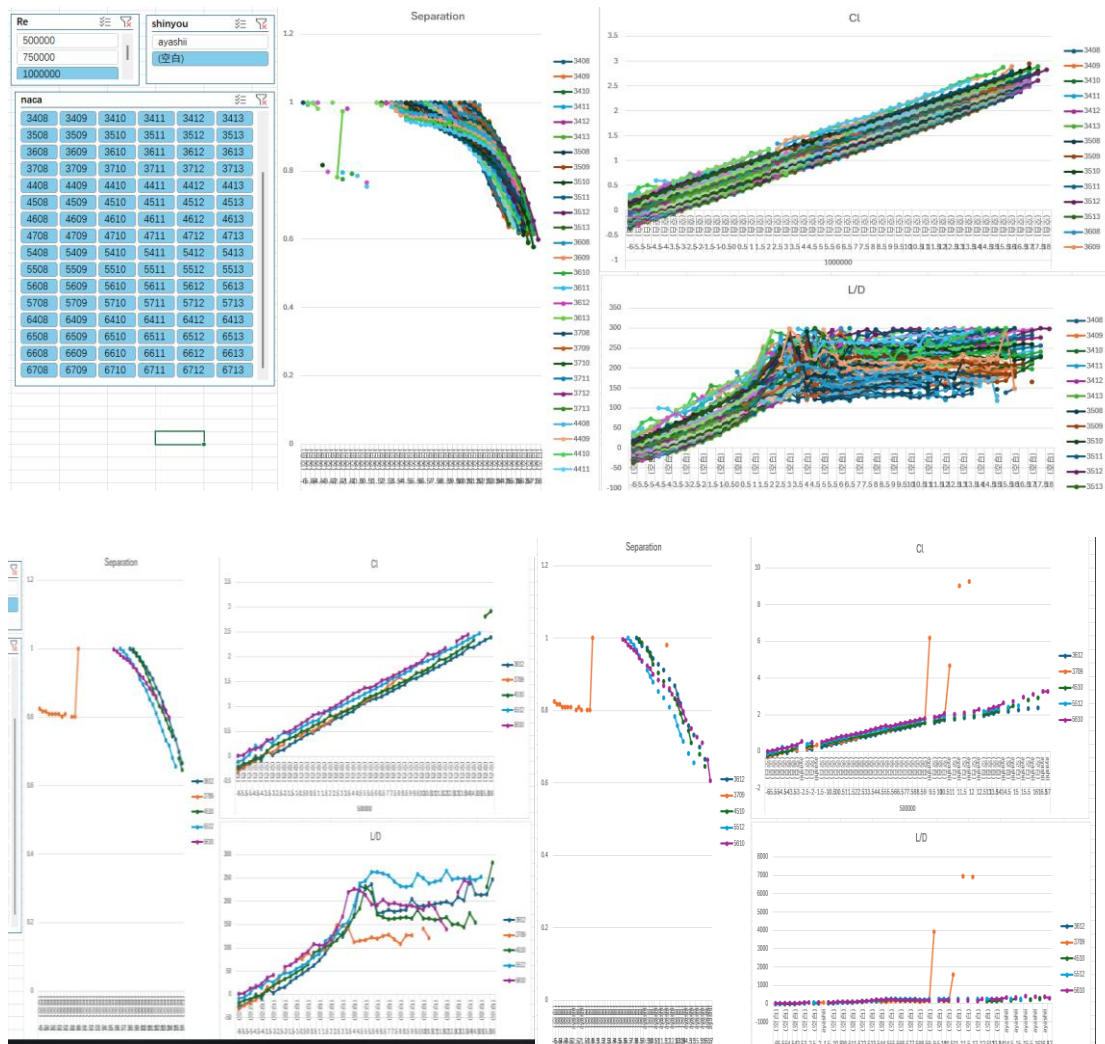
- **V2.0: Strict Physical and Numerical Validation (Airfoil database 1.2, 1.3)**

To solve the "Garbage In, Garbage Out" problem, I established a multi-layer validation system:

- Separation Point Monitoring: Detecting premature stall.
- Threshold Filtering: Removing coefficients ( $C_L$ ,  $C_D$ ) outside physical bounds.
- Gradient Checks: Identifying numerical divergence via sudden  $L/D$  jumps between adjacent angles of attack.

```
#Translations: "jyoutai" means "status" in Japanese, "ayashii" means "dodgy", "shinyou" means "trust/credit"
jyoutai = None
if output_results:
    #Stall monitor: TE stall after x = 0.7 is acceptable, LE stall leads to end of calculation of current airfoil
    if output_results["separation"]:
        if output_results["separation"] <= 0.7:
            stall_counts[nameae] += 1
        if output_results["separation"] < 0.1:
            stall_counts[nameae] = 3
            jyoutai = "ayashii"
    #Physically unrealistic quantities
    if output_results["cl"] >= 3 or output_results["cd"] >= 0.02 or output_results["ld"] >= 300:
        stall_counts[f"{nameae}_wierd_reading"] += 1
        jyoutai = "ayashii"
    #check sudden changes (instead of a gradual process) of L/D, which indicates xfoil is doing sth wierd
    if len(results_buffer) != 0:
        delta_ld = (-results_buffer[len(results_buffer)-1]["ld"] + output_results["ld"])/abs(results_buffer[len(results_buffer)-1]["ld"])
        #print(f"delta l/d: {delta_ld}, shinyou: {jyoutai}")
        # We only track sudden increase, as it might just be a vibration. If it dropps dramatically and rises again it will still be detected
        if delta_ld >= 0.4 and results_buffer[-1]["naca"] == naca and abs(results_buffer[len(results_buffer)-1]["ld"]) >= 40:
            stall_counts[f"{nameae}_wierd_reading"] += 1
            jyoutai = "ayashii"
    print(f"delta l/d: {delta_ld}, shinyou: {jyoutai}\n")
```

The result was a high-quality NACA database with minimal noise, ready for machine learning.



Filtered (left) vs Unfiltered (right) data

P

### hase III: Parametric Airfoil Tools

- V1.0: Linear VBA Interface**

Inspired by AI-assisted CAD, I developed a Python-to-SolidWorks interface that converts CSV coordinates into VBA macros. The initial version used linear segments, which lacked the smoothness required for aerodynamics, and were soon replaced by splines.

- V2.0: Airfoil Class & CST Parameterization**

I encapsulated generation, transformation, and fitting functions into an Airfoil Class. Replacing the unstable PARSEC method, I introduced CST (Class Shape Transformation) parameterization. Using the Least Squares Method, I solved the matrix for optimal weights of the Bernstein polynomials. This ensured smooth, natural geometries with standardized edges.

- **V3.0: XFOIL-style CLI**

I developed a command-line interface (CLI) that mimics XFOIL's interaction logic, integrating generation, CST fitting, plotting, and CSV/VBA exporting for rapid iteration.

```
=====
AIRFOIL TOOLBOX INTERACTIVE TERMINAL
=====
Current Status: Empty
-----
1. [NACA] Generate NACA 4-Digit Profile
2. [Load] Load Airfoil from CSV
3. [Tran] Transform (Scale/Rotate) & Export
4. [VBA] Generate SolidWorks VBA Macro
5. [Fit] CST Fitting (Comparison & Auto-Export)
6. [Plot] Visualize Current Profile
7. [CST] Generate Airfoil from Weights
8. [Flip] Flip & Export Flipped CSV (for Downforce)
q. [Exit] Quit
-----
Select an option: |
```

- **V4.0: High-Camber Compatibility**

I refactored the logic for identifying upper/lower surfaces. Instead of relying on y-axis polarity (positive for upper surface and negative for lower, which fails for high-camber airfoils where  $y_{lower}$  could be positive),

```
# 1. 筛选并去重 (保留 y=0 的阈值)
upper_df = df[df['y'] >= 0].drop_duplicates(subset=['x', 'y'])
lower_df = df[df['y'] <= 0].drop_duplicates(subset=['x', 'y'])
```

the tool now splits data based on the trailing edge ( $x = 1$ ) and identifies surfaces via regional means.

```
df = pd.read_csv(file_path)
# 确保数据为浮点数
pts = df[['x', 'y']].values.astype(float)

# 2. 寻找第一个 x 接近 1.0 的点
# 注意: 由于浮点数精度, 用 np.isclose 或设定阈值更稳妥
te_start_idx = np.where(np.isclose(pts[:, 0], 1.0, atol=1e-3))[0][0]

# 3. 按照你的逻辑, 从序列开始到后缘点, 和从后缘点到结束进行切分
# 通常 side1 是 Upper (如果序列是 TE-UP-LE), side2 是 Lower (LE-LO-TE)
side1 = pts[:te_start_idx + 1]
side2 = pts[te_start_idx:]

# 4. 自动识别并重排: 我们要的结果是 self.upper/lower 都是从 LE 到 TE 且 x 递增
if np.mean(side1[:, 1]) > np.mean(side2[:, 1]):
    up_raw, lo_raw = side1, side2
else:
    up_raw, lo_raw = side2, side1
```

I also added features for airfoil flipping (for racecard of course), mean camber line plotting, and max camber display.

---

## Phase IV: Interactive Visualization Platform

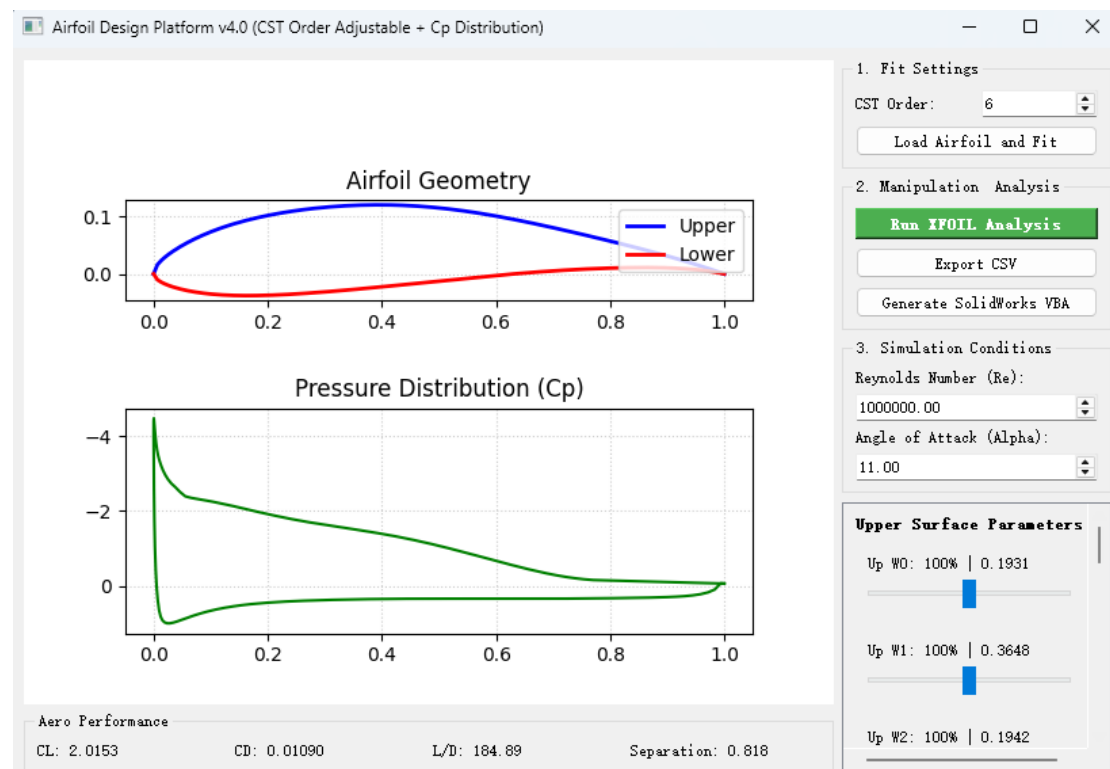
- **V1.0: PyQt5 Base GUI**

Built a graphical interface using PyQt5 and integrated Matplotlib for real-time rendering. Users could adjust CST weights via sliders and export to CSV or VBA, though it lacked aerodynamic feedback.

- **V2.0: Design-Simulation Closed-Loop Platform**

I integrated the automated XFOIL scripts from Phase I/II using multithreading (asynchronous) to prevent UI freezing.

Full Workflow: Import CSV/NACA - CST fitting - Real-time slider adjustment - Asynchronous 2D simulation - Visualization pressure distribution ( $C_p$ ) - Engineering export. This created a complete loop from parametric design to aerodynamic validation.

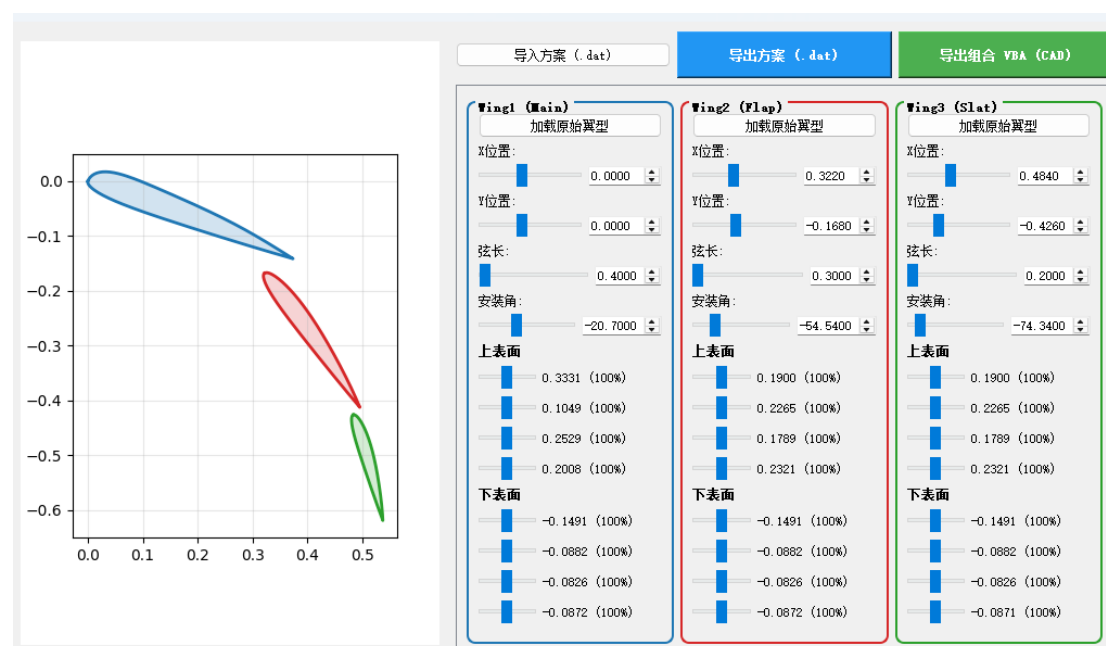


- **V3.0: Multi-Element Support & Logging System**

```
design_log_20260510_1501.csv
design_log_20260510_1502.csv
design_log_20260510_1505.csv
design_log_20260510_1512.csv
design_log_20260510_1521.csv
design_log_20260511_1853.csv
design_log_20260511_2139.csv
```

**Multi-Element Capability:** Added visualization and adjustment (position, angle, scale) for up to three independent wing segments to support FSUK multi-element designs.

Industry Standards: Migrated all I/O formats to .dat to align with professional aerodynamic standards (e.g., UIUC database)



## V. Project Outcomes and Technical Value

This project solved core technical hurdles including XFOIL stability, data quality, and parametric precision. By transitioning from empirical hand-sketching to data-driven design, the workflow now covers Python automation, XFOIL API development, data cleaning, and UI design. This provides a robust foundation for future airfoil optimization, machine learning, and 3D wing design, significantly enhancing design efficiency and reproducibility.

---

## VI. Future Roadmap & Strategic Goals

### 1. Deep Learning & Inverse Design

- **Predictive Modeling:** Leveraging the high-quality NACA and CST databases generated in Phases I & II to train a neural network capable of predicting aerodynamic coefficients ( $C_L$ ,  $C_D$ ,  $C_m$ ) in milliseconds.
- **Inverse Design:** Developing a Generative Adversarial Network (GAN) or Variational Autoencoder (VAE) to perform inverse design—generating the optimal airfoil geometry based on target aerodynamic performance parameters.

### 2. High-Fidelity CFD Integration

- **Panel Method Solver:** Developing an integrated potential flow solver (Panel Method) within the platform for instantaneous, inviscid performance estimates of multi-element configurations.
- **OpenFOAM Automation:** Establishing a 2D OpenFOAM automated meshing and simulation pipeline for multi-element wings. This will allow for the study of complex viscous effects, such as wake-flap interaction and flow separation, providing higher fidelity data while saving 3D CFD computational resources.

### Additional Statement:

*While AI assisted in generating low-level code, syntax implementation, and PyQt5 interaction logic, I independently managed the technical selection, overall software architecture, functional modularization, data cleaning rules, parametric algorithm design, and iterative debugging. I maintained full control over the technical roadmap and engineering implementation throughout the project.*